Algorithms

Divide and Conquer

Dr. Mudassir Shabbir

LUMS University

March 16, 2026

Dynamic Programming

- Solve optimization problems using **overlapping subproblems**.
- Store solved subproblems to avoid recomputation.
- Typical gain: exponential recursion $\rightarrow$ polynomial/linear time.

When DP Applies + Design Pattern

Two required properties

- **Optimal substructure:** optimal solution is built from optimal subproblems.
- **Overlapping subproblems:** same subproblems repeat many times.

DP recipe

1. Define a state (what does DP entry represent?).
2. Write a recurrence relation.
3. Set base cases.
4. Compute with memoization (top-down) or tabulation (bottom-up).

Note: If subproblems do not overlap, Divide & Conquer is usually better.

Fibonacci: Why DP Helps

Naive recursion:

$$\mathsf{Fib}(n) = \mathsf{Fib}(n-1) + \mathsf{Fib}(n-2), \quad T(n) = O(2^n)$$

Repeated calls to the same subproblems cause exponential blow-up.

DP improvement

- **Top-down (memoization):** cache each $\mathsf{Fib}(i)$ once.
- **Bottom-up (tabulation):** fill $\mathrm{DP}[0 \ldots n]$ iteratively.

Both give $O(n)$ **time**. Space is $O(n)$ (or $O(1)$ with rolling variables).

Example 1: Coin Change (Min Coins)

Problem: coins $[c_1, \ldots, c_k]$, target n. Find minimum coins.

State: $\mathrm{DP}[i]$ = minimum coins to make amount i.

Recurrence:

$$\mathrm{DP}[i] = \min_{c \in \text{coins},\ c \le i} (\mathrm{DP}[i-c] + 1), \quad \mathrm{DP}[0] = 0.$$

Complexity: $O(nk)$ time, $O(n)$ space.

For coins $[1, 2, 5]$, target 11: answer is **3** $(5 + 5 + 1)$.

Example 2: LCS (Longest Common Subsequence)

State: $\mathsf{LCS}(i, j) = $ LCS length of prefixes $X[1..i]$, $Y[1..j]$.

Recurrence:

$$\mathsf{LCS}(i,j) = \begin{cases} 1 + \mathsf{LCS}(i-1, j-1), & X[i] = Y[j], \\ \max\{\mathsf{LCS}(i-1, j), \mathsf{LCS}(i, j-1)\}, & X[i] \neq Y[j]. \end{cases}$$

Base cases: $\mathsf{LCS}(0, j) = \mathsf{LCS}(i, 0) = 0$.

DP table size is $(m+1) \times (n+1)$, so $O(mn)$ **time** and $O(mn)$ space.

LCS Example: Final DP Table

Example: $X = \texttt{ABCB}$, $Y = \texttt{BDCAB}$

	ε	B	D	C	A	B
ε	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	**3**

Answer: LCS length is **3**. One LCS is BCB.

Example: The Gold-Extractor Game

Problem: Coins are hidden in an $n \times n$ integer grid.

- A player controls two extractors:
  - **Vertical** extractor (red) moves right by one column line.
  - **Horizontal** extractor (blue) moves up by one row line.
- At each step, move exactly one extractor by one line.
- If the new line contains coins, collect exactly one; all other coins on that same line are lost.
- The game ends when *either* extractor reaches its final line.

Example: The Gold-Extractor Game

Problem: Coins are hidden in an $n \times n$ integer grid.

- A player controls two extractors:
  - **Vertical** extractor (red) moves right by one column line.
  - **Horizontal** extractor (blue) moves up by one row line.
- At each step, move exactly one extractor by one line.
- If the new line contains coins, collect exactly one; all other coins on that same line are lost.
- The game ends when *either* extractor reaches its final line.

Goal

Choose *which* extractor to move at each step to maximize total coins collected.

A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.

A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.

A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.

A Combinatorial Game

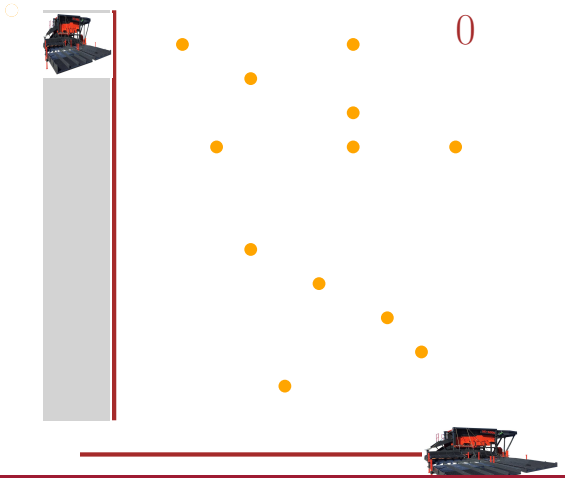
- Goal is to maximize the profit by carefully choosing which machine to move at each step.

A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.

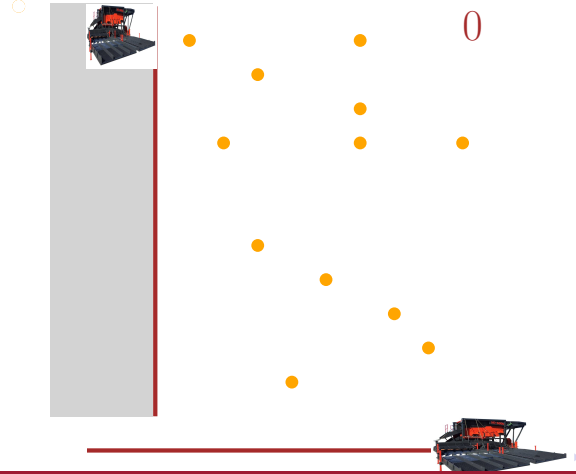
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



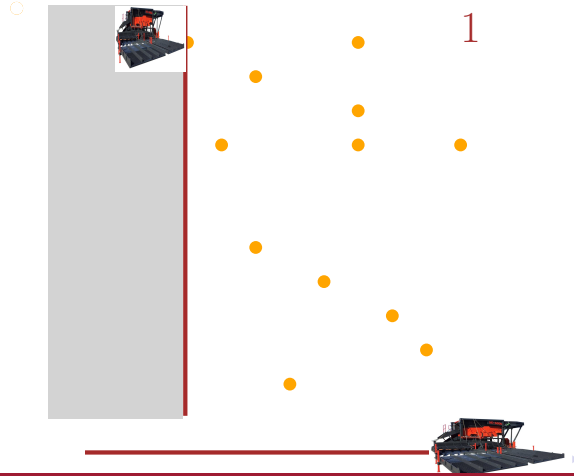
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



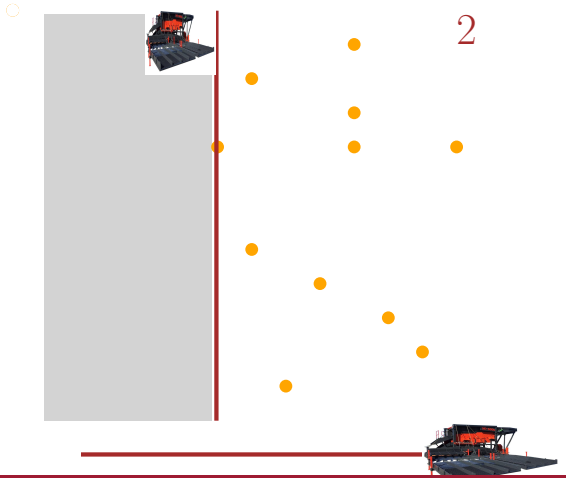
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



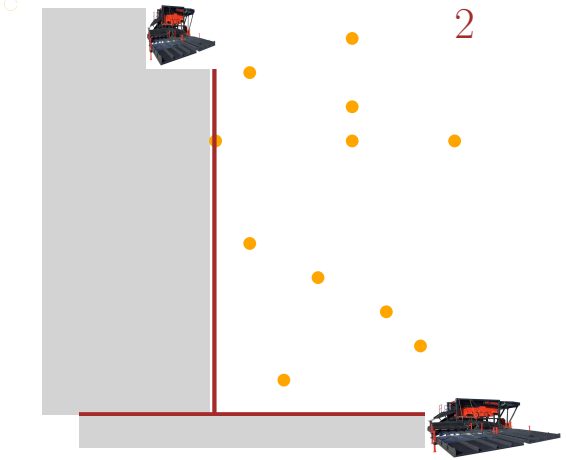
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



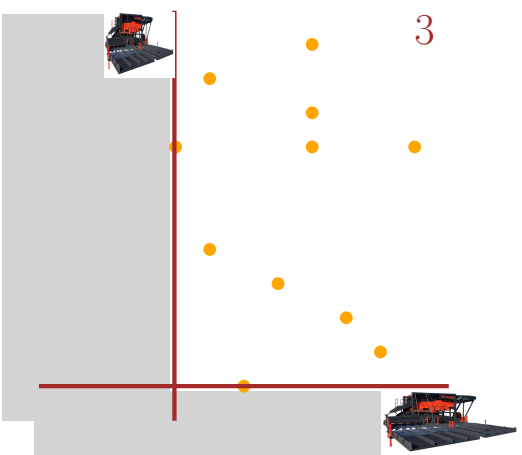
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



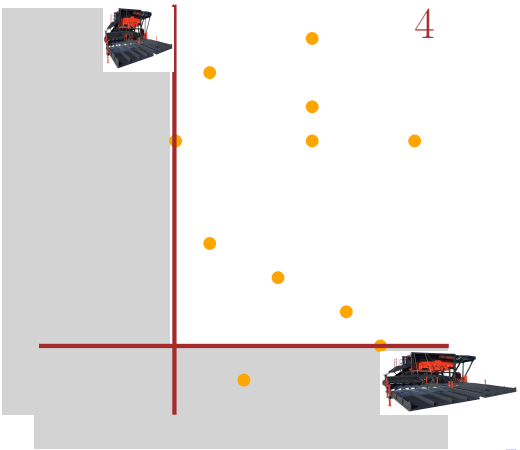
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



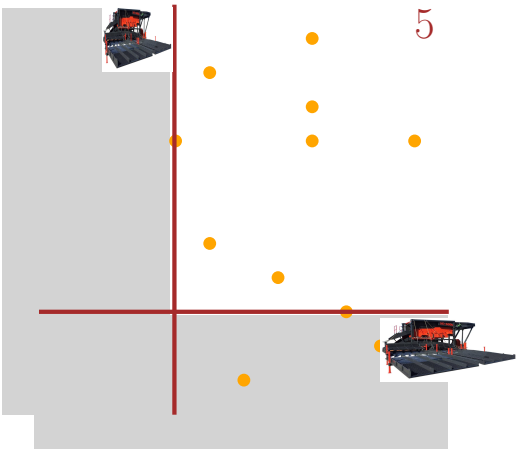
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



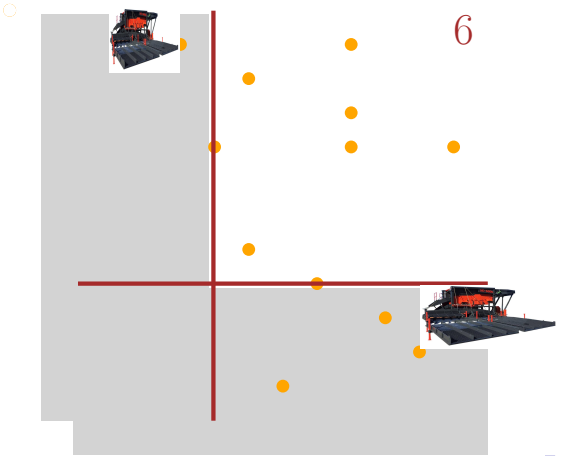
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



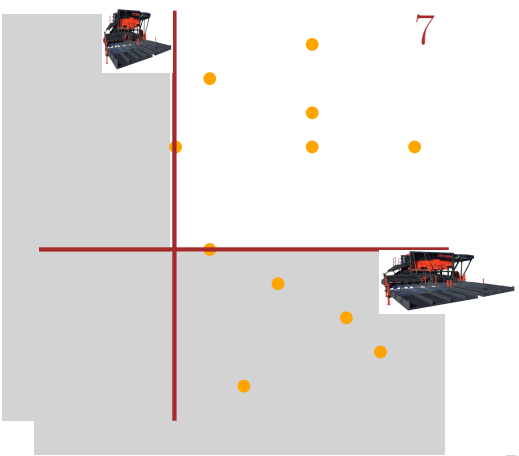
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



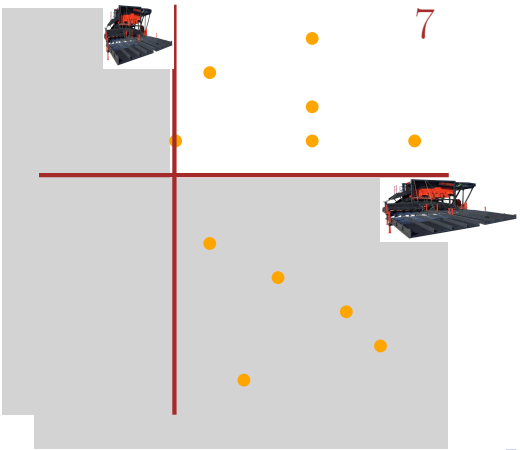
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



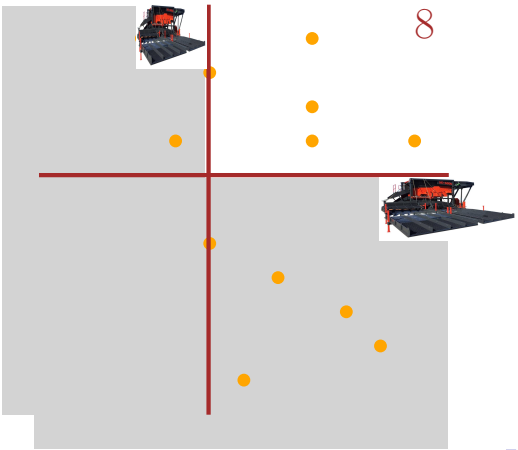
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



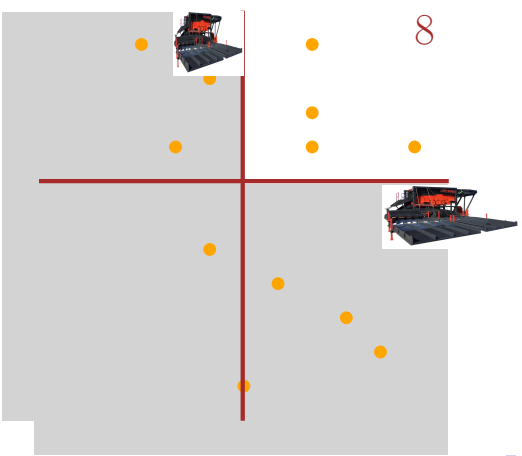
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



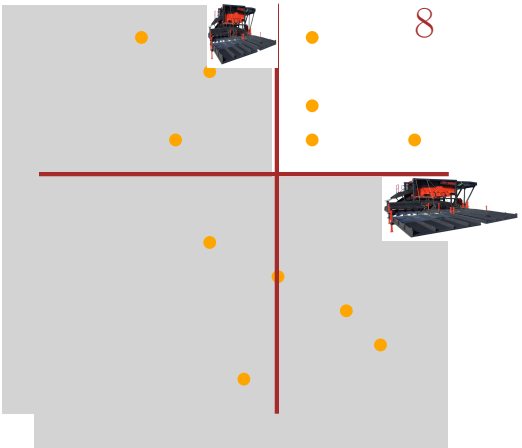
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



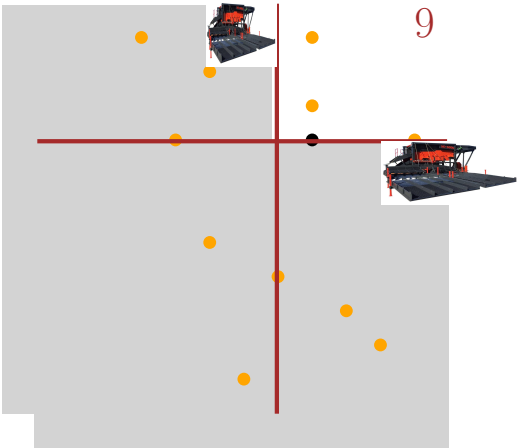
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



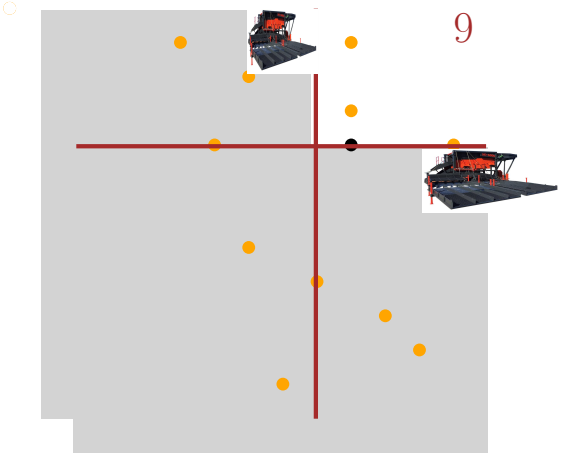
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



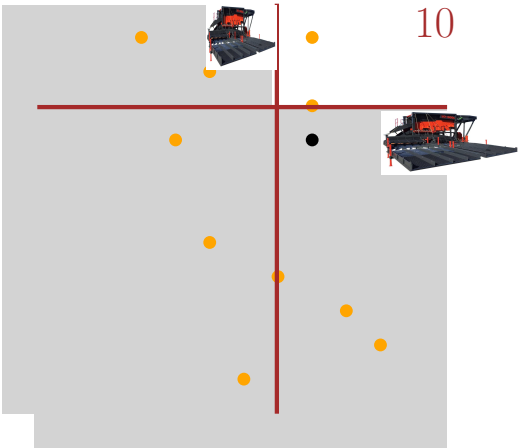
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



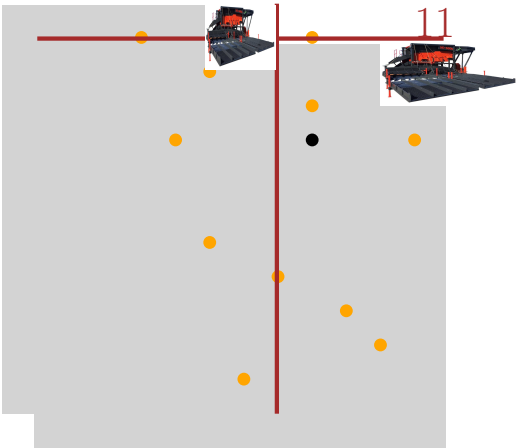
A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.



A Combinatorial Game

- Goal is to maximize the profit by carefully choosing which machine to move at each step.

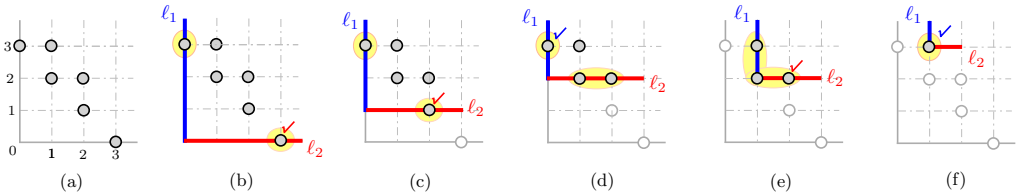


Greedy Approach

Greedy idea: At each step, advance the machine whose next position has *fewer* coins — preserving the richer direction for later.

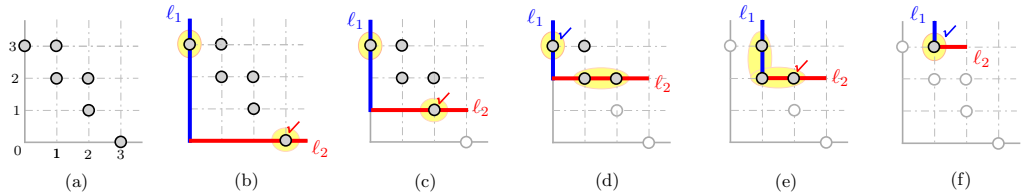
Greedy Approach

Greedy idea: At each step, advance the machine whose next position has *fewer* coins — preserving the richer direction for later.



Greedy Approach

Greedy idea: At each step, advance the machine whose next position has *fewer* coins — preserving the richer direction for later.



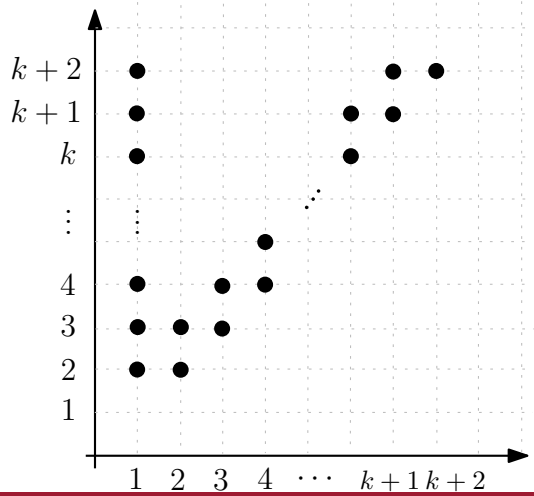
In practice, this heuristic performs well on random instances.

When Greedy Fails

However, greedy can perform poorly on adversarial inputs.

When Greedy Fails

However, greedy can perform poorly on adversarial inputs.



Dynamic Programming Solution

State: Let $\alpha(c, r)$ be the maximum coins collectible from the current position onward, when machine 1 is at **column** c and machine 2 is at **row** r.

Dynamic Programming Solution

State: Let $\alpha(c, r)$ be the maximum coins collectible from the current position onward, when machine 1 is at **column** c and machine 2 is at **row** r.

Recurrence: At state (c, r) we choose which machine to advance:

$$\alpha(c, r) = \max\Big(\underbrace{\alpha(c+1, r) + \mathbf{1}_c}_{\text{advance machine 1 (column)}} \quad , \quad \underbrace{\alpha(c, r+1) + \mathbf{1}_r}_{\text{advance machine 2 (row)}} \Big)$$

where $\mathbf{1}_c = 1$ if column c has a coin on row $r+$, $\mathbf{1}_r = 1$ if row r has a coin on column $c+$.

Dynamic Programming Solution

State: Let $\alpha(c, r)$ be the maximum coins collectible from the current position onward, when machine 1 is at **column** c and machine 2 is at **row** r.

Recurrence: At state (c, r) we choose which machine to advance:

$$\alpha(c, r) = \max\Big(\underbrace{\alpha(c+1, r) + \mathbf{1}_c}_{\text{advance machine 1 (column)}} \quad , \quad \underbrace{\alpha(c, r+1) + \mathbf{1}_r}_{\text{advance machine 2 (row)}} \Big)$$

where $\mathbf{1}_c = 1$ if column c has a coin on row $r+$, $\mathbf{1}_r = 1$ if row r has a coin on column $c+$.

Base case: $\alpha(n+1, r) = \alpha(c, n+1) = 0$ (machine has reached the end).
Answer: $\alpha(0, 0)$.

Dynamic Programming Solution

State: Let $\alpha(c, r)$ be the maximum coins collectible from the current position onward, when machine 1 is at **column** c and machine 2 is at **row** r.

Recurrence: At state (c, r) we choose which machine to advance:

$$\alpha(c, r) = \max\Big(\underbrace{\alpha(c+1, r) + \mathbf{1}_c}_{\text{advance machine 1 (column)}} \quad , \quad \underbrace{\alpha(c, r+1) + \mathbf{1}_r}_{\text{advance machine 2 (row)}} \Big)$$

where $\mathbf{1}_c = 1$ if column c has a coin on row $r+$, $\mathbf{1}_r = 1$ if row r has a coin on column $c+$.

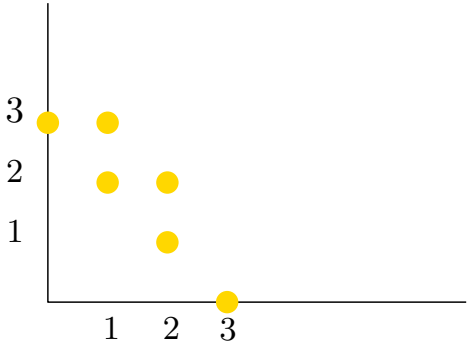
Base case: $\alpha(n+1, r) = \alpha(c, n+1) = 0$ (machine has reached the end).
Answer: $\alpha(0, 0)$.

Why DP?

The same state (c, r) is reachable via many different move sequences — **overlapping subproblems**. Storing $\alpha(c, r)$ avoids recomputation.

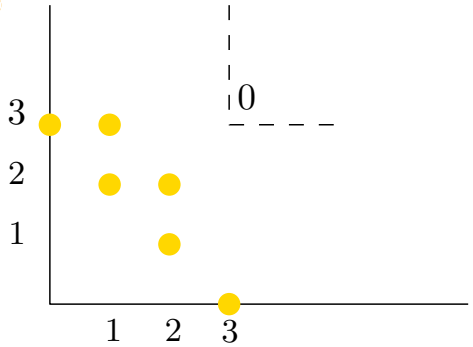
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



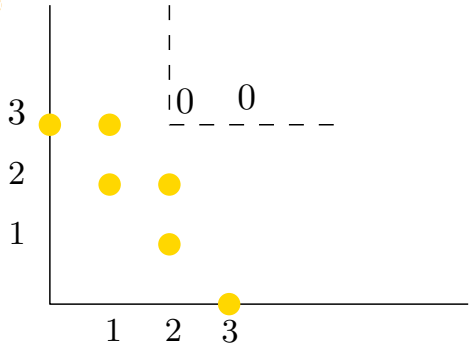
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



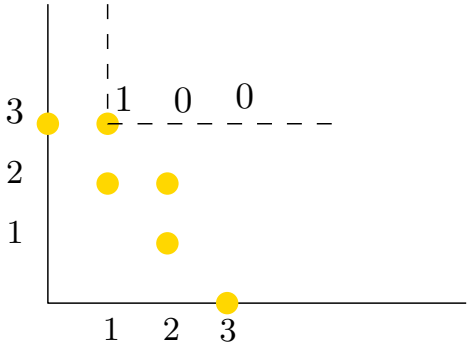
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



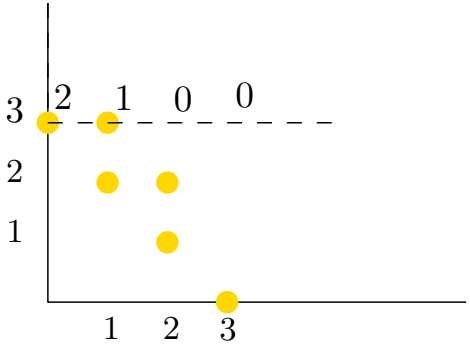
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



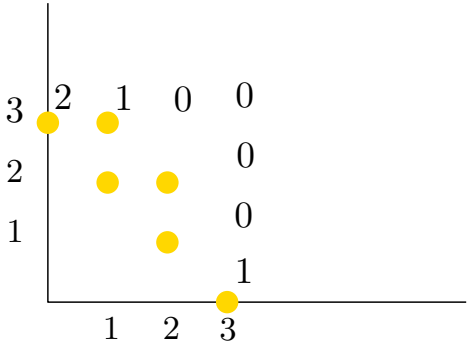
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



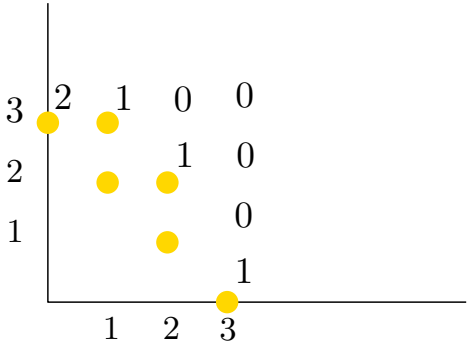
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



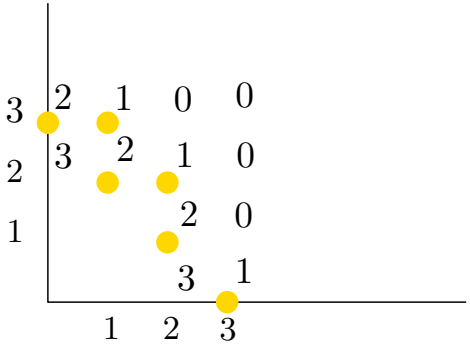
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



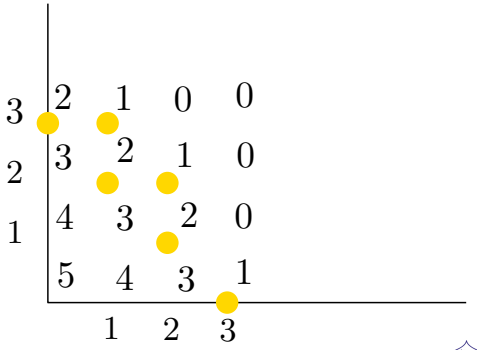
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



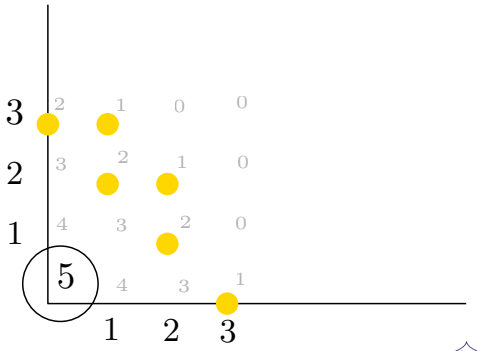
DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.



DP Solution: Filling the Table

- States (c, r) form an $n \times n$ table (column $\times$ row).
- Fill from the boundary (base cases at $c = n$ or $r = n$) toward $(0, 0)$.
- Each cell $\alpha(c, r)$ depends only on its right neighbor $\alpha(c + 1, r)$ and its top neighbor $\alpha(c, r + 1)$.

Time: $O(n^2)$ states, $O(1)$ work each $\Rightarrow O(n^2)$ total.